k-Means Clustering

Sean Hellingman ©

Multivariate Statistics for Applied Data Science (ADSC 4050)

shellingman@tru.ca

Winter 2026

THOMPSON RIVERS UNIVERSITY

Topics

Recap

- Last time we covered **Hierarchical Clustering**:
  - Introduced clustering.
  - Algorithm.
  - Dendrograms.
  - Linkage methods.
  - Selecting the number of groups.

Introduction

- We have examined hierarchical clustering.
  - There are many ways to adjust the algorithm.
- Again, we are trying to group our data into natural *clusters*.
- Another commonly used approach is k-means clustering.
  - Requires a selection of k groups prior to running.

k-Means Clustering

Introduction k-Means Clustering

- k-**means clustering** is an unsupervised, non-hierarchical technique that partitions the observations into k groups.
- The algorithm assigns objects to groups based on minimizing some criterion.
  - *This criterion may change depending on the clustering task.*
- Generally, we want to minimize the variance within groups.
  - How variance is calculated varies among algorithms.

Illustrative Example 1

1. Import the *iris* dataset into R.
2. Run the provided code to generate a 3D scatterplot.
3. What do you notice?
4. Which value(s) of k might you want to try?

Algorithm

- The iterative process progresses as follows:
  1. Select k starting locations (centroids).
  2. Assign each observation to the group to which it is closest.
  3. Calculate within-group sums of squares (or other criterion).
  4. Adjust coordinates of the k locations to reduce variance
  5. Re-assign observations to groups, re-assess variance.
  6. Repeat process until stopping criterion is met.

- Due to the nature of the algorithm, it is possible to only achieve a local minimum rather than a global minimum.
  - Due to differences in starting locations.

- Often, using multiple starts will increase the chances of finding a global minimum.

Illustrative Example 2

1. Import the *iris* dataset into R.
2. Run the provided code to generate the progression of the k-means clustering algorithm.
   - *Run it in your console for a moving animation.*
3. What do you notice?
4. Change the value of k and try again.

Using R

- There are many R packages that allow you to perform this analysis.
  - In Python
- *We will cover some of the functions.*

Base R

- We can use:

```
kmeans(x, centers, iter.max = 10, nstart = 1, algorithm
= "Hartigan-Wong", trace=FALSE)
```

  - `x` is a numeric data matrix (not a distance matrix).
  - `centers` is the number of clusters to form (k).
  - `nstart` is the number of random sets of starting points to choose.

- Note: This approach minimizes the sum of squared (Euclidean) distances.

Example 1

- Perform k-means clustering on the iris dataset for the following number of groups:
  1. $k = 2$
  2. $k = 3$
  3. $k = 4$
- What happens when you re-run the code?
- Take some time to interpret the results.

Random Starts

- It is possible to only achieve a local minimum rather than a global minimum.
- In order to mitigate this problem, we should use *multiple starts.*
  - Meaning we run the algorithm multiple times from different starting points.
- *Remember to set your seed for reproducible results!*

Example 2

- Perform k-means clustering on the iris dataset for the following number of groups **and increase the number of random starts**:
  1. $k = 2$
  2. $k = 3$
  3. $k = 4$
- Did this change your results at all?
- Take some time to interpret the results.

Cluster Quality

- The *quality* of a cluster is measured by its within-cluster sum-of-squared-distances (WSSD).
- The **WSSD** is the sum of the squared distances of each object in the cluster from the centroid of the cluster.
- The larger the value, the more spread out that cluster is.
  - *This is relative to the scale of the variables and the number of observations in the cluster.*
- Summing all of these values together gets the **total WSSD**.

Number of Clusters

Number of Clusters (WSSD)

- We can use the within-cluster sum-of-squared-distances (WSSD) to get an idea of how many clusters to select.
- Generally, simpler (fewer clusters) is easier to interpret.
  - Important to find the balance between interpretability, sample size, and task objectives.
- One way to select the number of clusters is the elbow method.
- Plot the WSSD by the number of clusters and look for the *elbow*.

Example 3

- Run the provided code to determine how many clusters to select based on the WSSD.
- What happens when you change some of the arguments?
- What do you notice?

Number of Clusters (CHI)

- We can use the **Calinski–Harabasz index** (CHI) to get an idea of how many clusters to select.
  - Higher values indicate a better performance.
- This can be achieved in R using the vegan package.
- Usage:

```
cascadeKM(data, inf.gr = min, sup.gr = max)
```

- *Plotting the results will give you a more concise visualization.*

Example 4

- Use the `cascadeKM()` function to determine the ideal number of groups based on the CHI.
- Plot your results.
- What do you notice?

More flexible Approaches

- The methods that we have covered typically utilize euclidean distance and require numeric variables.
- The `pam()` function from the *cluster* package allows for more flexibility in the approach.
- The `clara()` function from the *cluster* package allows for effective clustering of large datasets through sampling.

Fuzzy Clustering

Fuzzy Clustering

- Both of the clustering approaches we have used so far, assigns the objects to a cluster.
- **Fuzzy clustering** determines a probability that each object belongs to a certain cluster.
- *Some sample objects might clearly belong to one group whereas others are marginal.*

Fuzzy Clustering in R

- We can use the `fanny()` function from the *cluster* package to do this.
- Usage:

```
fanny(data,k=clusters)
```

- The output will give you probabilities of the object belonging to each group.
  - You can also plot the results to see the *overlap* in the clusters.
- *Note: This algorithm can take a dissimilarity matrix instead of the dataset (see documentation).*

Example 5

- Perform fuzzy clustering on the iris dataset for the following number of groups:
  1. $k = 2$
  2. $k = 3$
  3. $k = 4$
- What happens when you re-run the code?
- Take some time to interpret the results.

Comments

- **Cluster analysis does not test hypotheses.**
  - You can test hypotheses or perform analyses using the clusters as a grouping variable.
- Most of the approaches we have taken in this lecture use Euclidean distances.
  - **Meaning that the scale of the variables will impact the results.**
- I suggest using multiple methods to ensure that your results are consistent.
- *This is not an exhaustive approach to clustering in k groups.*

Next Lecture

- We will discuss **Classification**.
- *Supervised learning*
- Discriminant Analysis

Exercise 1

- Import the wine dataset from the *HDclassif* package.
- Take some time to get to know the data (including visualizations).
- Follow the steps that we have covered in these slides to perform multiple k-means clustering analyses.
- Does your confidence in these results change with fuzzy clustering?
- Note: The first column `class` is the group identifier and should not be included in your analysis.
  - Because you have the `class` variable, you can check your work.

References & Resources

1. Bakker, J. D. (2026) Applied Multivariate Statistics in R
2. Peng R. D. (2020) *Exploratory Data Analysis with R*
3. Timbers T., Campbell T., and Lee M. (2024) *Data science: A first introduction.* Chapman and Hall/CRC

- kmeans()
- cascadeKM()
- `pam()`
- `fanny()`